



S.G. Ganesh

Calling Virtual Functions from Constructors

Calling virtual functions from constructors is problematic, and this problem can manifest itself in many ways. In this column, we'll take a look at this problem, with specific examples.

Last year, I bought a BlackBerry mobile. It came with software that can be installed on a PC, with which one can transfer songs, data, etc, from the PC to the mobile. When I installed the software and started it, it promptly crashed with the exception: *pure virtual function call*! Surprisingly, over a period of five years, I've faced the same problem many times, and some of the screen-shots I've taken from different software are shown in Figures 1 to 3.

Note that this behaviour is not specific to Windows software; software compiled with GCC on Linux will fail with a similar exception. Now, let us dig deeper, to understand this software bug.

Virtual functions are resolved based on the runtime type of the object on which they are called. If we try invoking virtual methods from constructors or destructors, it is problematic. Why? Consider the case of calling a virtual function from a constructor. When the base class constructor executes, the derived object is not constructed yet. If there is a virtual function call that is supposed to bind to the derived type, how can it be handled?

The ways in which OO languages handle this situation differ. In C++, the virtual function is treated as non-virtual, and the base type method is called. In Java, the call is resolved to the derived type. Both these approaches can cause unintuitive results. Let us first discuss the C++ case, and then move on to the Java case.

In C++, if you try the following program, it will print *"Inside base::vfun"* since the virtual function is resolved to the base type (i.e., static type itself, instead of the dynamic type):

```
struct base {
    base() {
```

```
        vfun();
    }
    virtual void vfun() {
        cout << "Inside base::vfun\n";
    }
};
struct deri : base {
    virtual void vfun() {
        cout << "Inside deri::vfun\n";
    }
};
int main(){
    deri d;
}
```

Now, how about this program:

```
struct base {
    base() {
        base * bptr = this;
        bptr->bar();
        // even simpler ...
        ((base*)(this))->bar();
    }
    virtual void bar() =0;
};
struct deri: base {
    void bar(){ }
};
int main() {
    deri d;
}
```

Now, you'll get the *"pure virtual function call"* exception thrown by the C++ runtime, which is similar to the three screen-shots we saw earlier! In this case, the *bar()* method is a pure virtual function, which means that it is not